ANNA UNIVERSITY

MIT CAMPUS

PROJECT REPORT

ON

Word to PDF Converter

Student Name	V.Hitesh
Roll Number	2024503517
Subject	IOT Project
Department	Computer Science
Technology Stack	Node.js, Express.js, HTML, CSS, JavaScript, Multer, Mammoth, Puppeteer
Academic Year	2025-2026
Date	March 4, 2026

CONTENTS

Section	Page No.
1. Abstract	**3**
2. Introduction	**3**
3. Problem Statement	**3**
4. Objectives	**3**
5. Technology Stack	**3**
6. System Architecture	**3**
7. Implementation Details	**4**
8. Workflow	**4**
9. API/Module Description	**4**
10. Testing and Results	**4**
11. Screenshots with Explanations	**5-8**
12. Challenges and Limitations	**8**
13. Future Enhancements	**8**
14. Conclusion	**8**
15. References	**8**

1. Abstract

This project implements a web-based Word to PDF Converter that allows users to upload .doc or .docx files and receive downloadable PDFs quickly. The solution is designed for simple interaction, clean output, and practical usage in academic and professional scenarios.

2. Introduction

Word documents are easy to edit, while PDFs are ideal for sharing. This project bridges that gap by providing a browser-accessible converter that handles document upload, conversion, and download in a single workflow.

3. Problem Statement

Many conversion tools require local software installation, subscriptions, or complex steps. The objective is to create a lightweight online utility that converts standard Word files to PDF efficiently.

4. Objectives

- Accept .doc/.docx uploads and convert them to PDF.
- Provide drag-and-drop plus file browser upload modes.
- Show clear conversion feedback and downloadable output.
- Maintain reliable backend processing and cleanup.

5. Technology Stack

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js, Express.js
- Upload handling: Multer
- Word parsing: Mammoth
- PDF rendering: Puppeteer

6. System Architecture

Client sends document to the server, the server validates input, Mammoth converts Word to HTML, Puppeteer renders HTML to PDF, and the PDF is returned as download.

7. Implementation Details

- Unique filenames are generated to avoid collisions.
- Type and size validations prevent invalid uploads.
- PDF is generated in A4 format with print-friendly margins.
- Temporary files are cleaned after conversion.

8. Workflow

- Open the converter page.
- Select or drag a Word file.
- Click "Convert to PDF".
- Receive and open the generated PDF.

9. API/Module Description

Endpoint: POST /api/convert

Input: multipart/form-data with field `file`

Output: downloadable PDF or structured error response.

10. Testing and Results

- Upload and selection behavior verified.
- Conversion requests processed successfully.
- PDF download confirmed and output reviewed.

11. Screenshots with Explanations (Page 1)

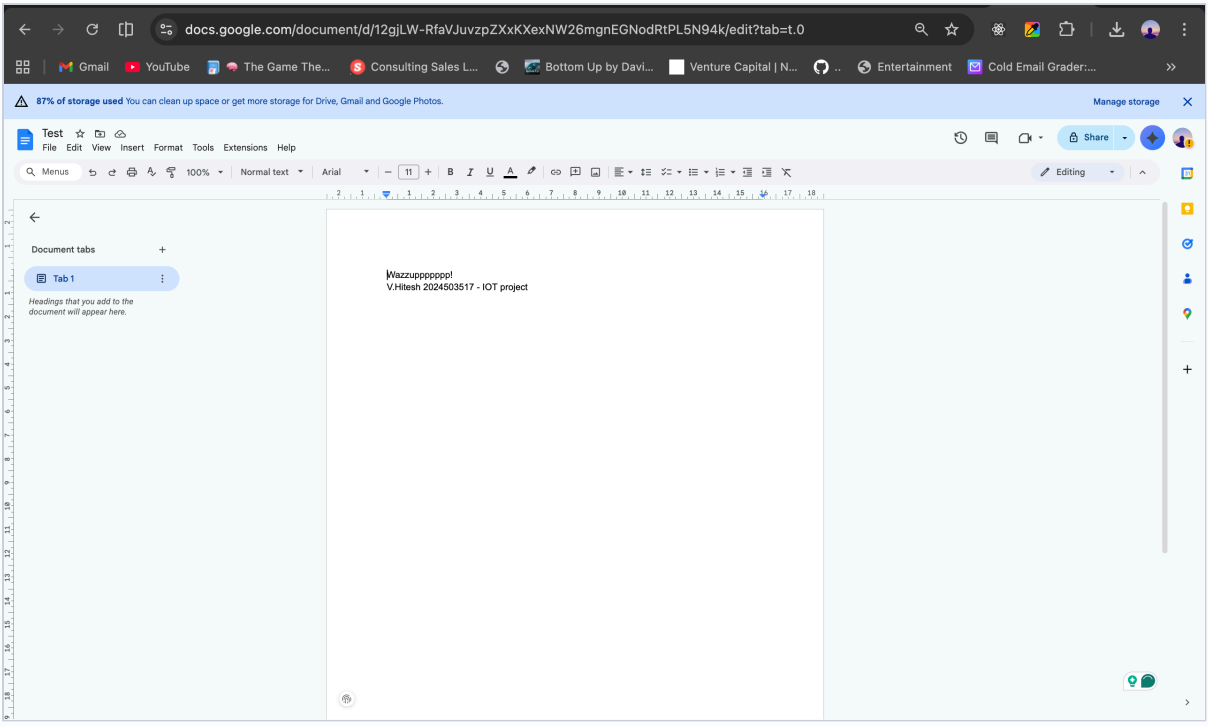

Figure 1: Home interface of the Word to PDF Converter.

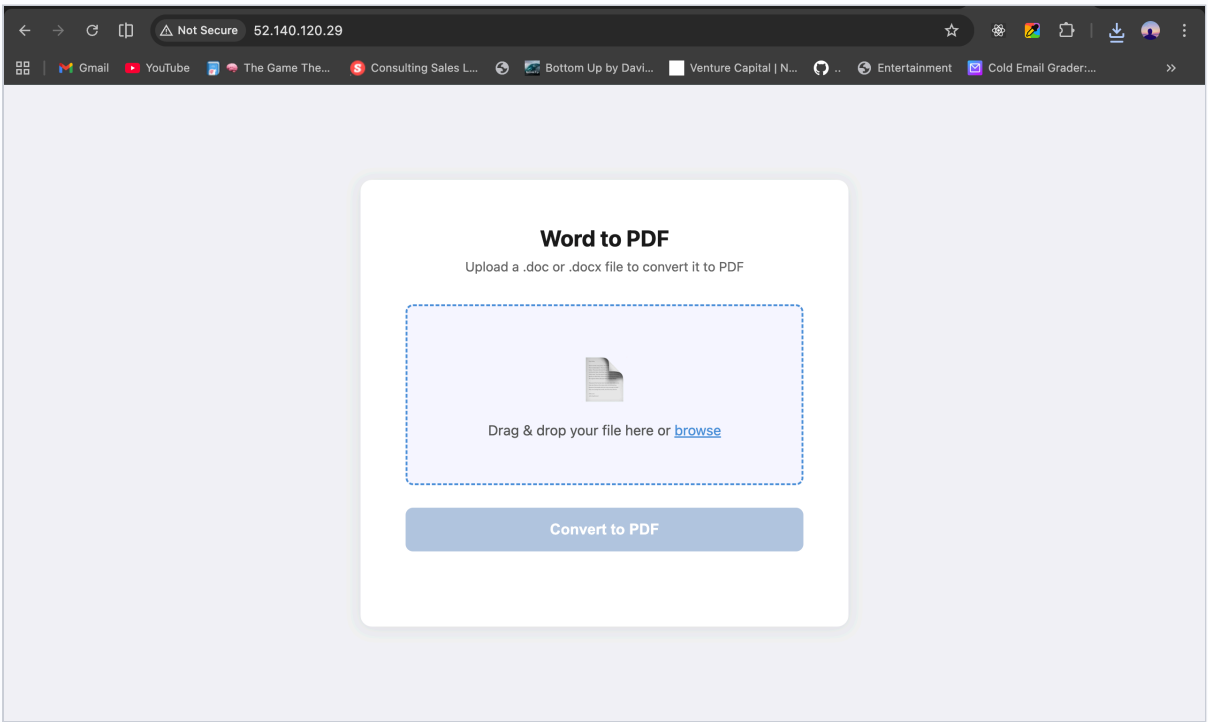

Figure 2: Sample source content prepared before conversion.

11. Screenshots with Explanations (Page 2)

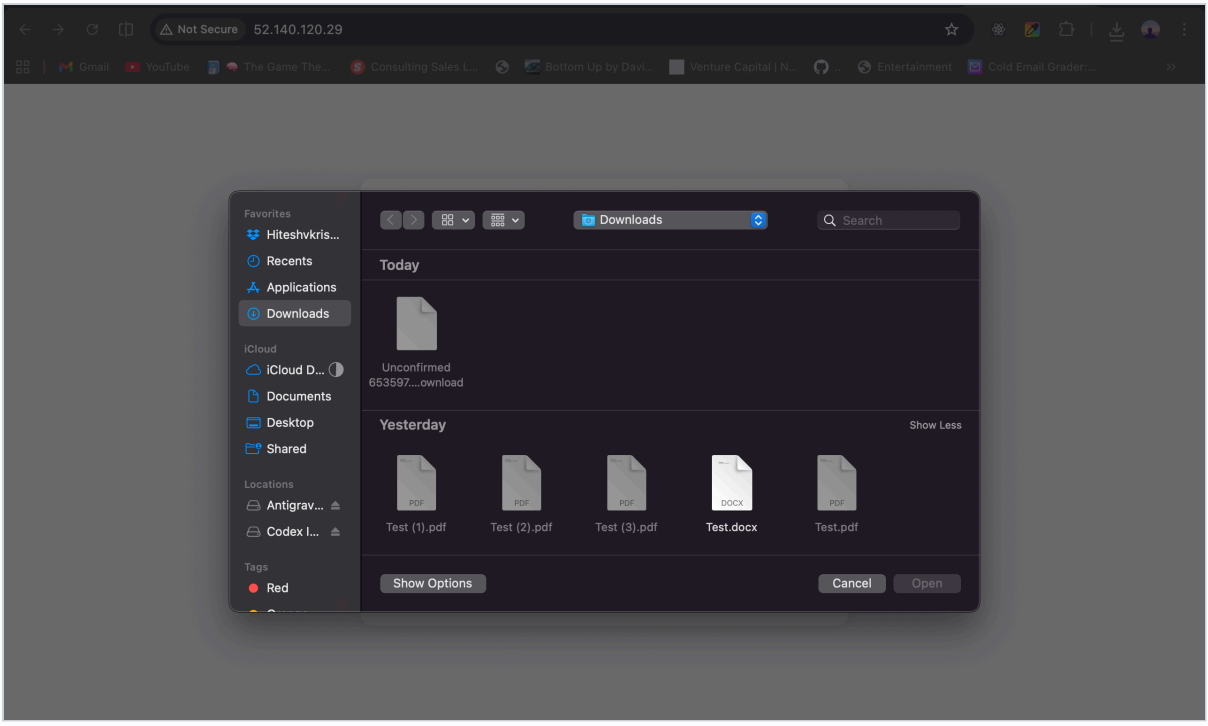

Figure 3: File picker dialog used to select .docx input.

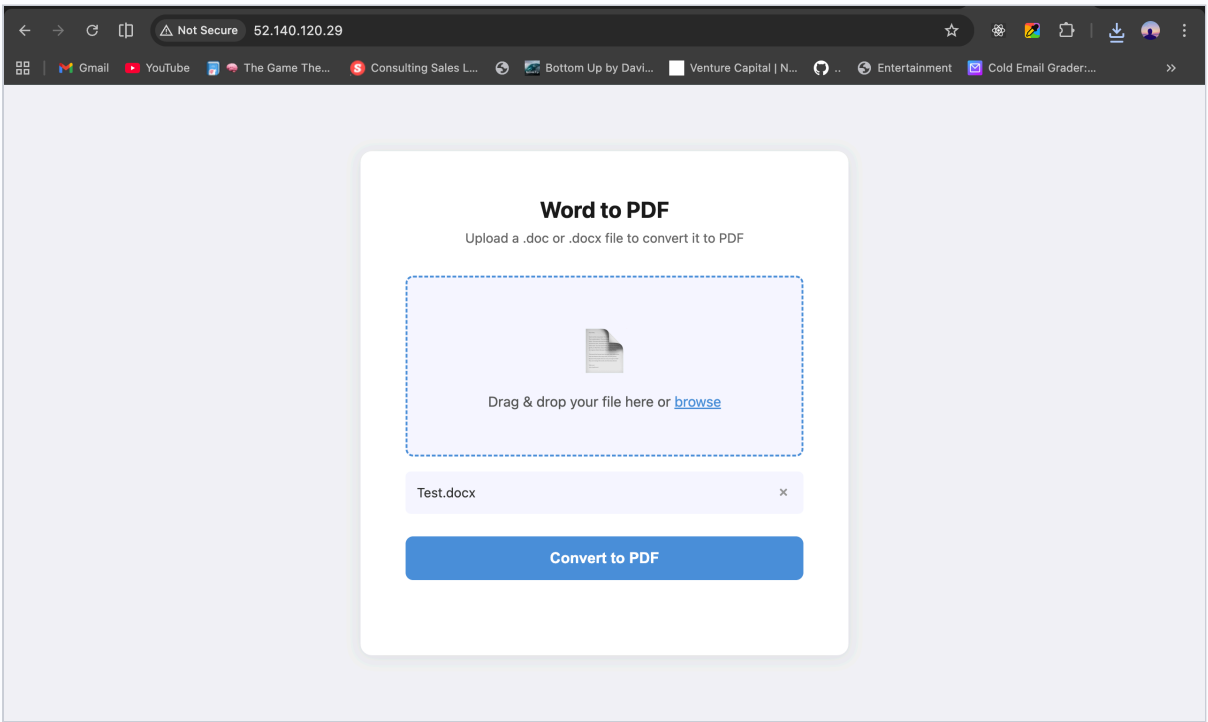

Figure 4: Selected file displayed and ready for conversion.

11. Screenshots with Explanations (Page 3)

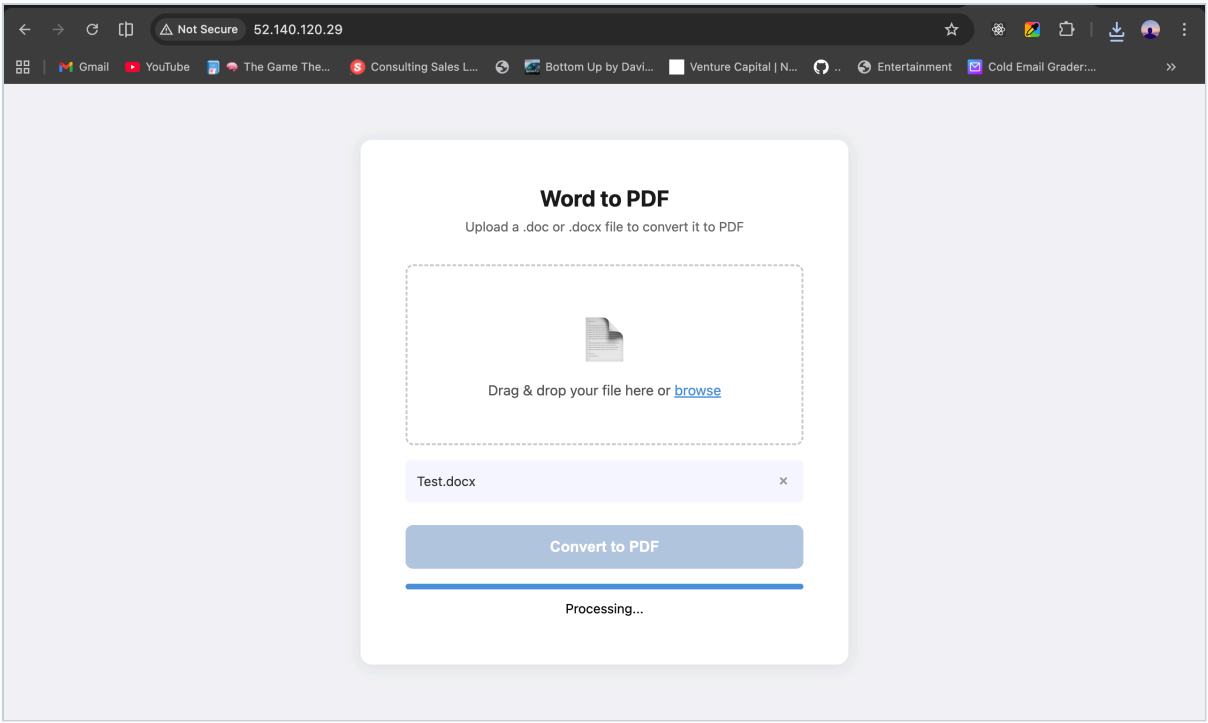

Figure 5: Successful conversion status shown after processing.

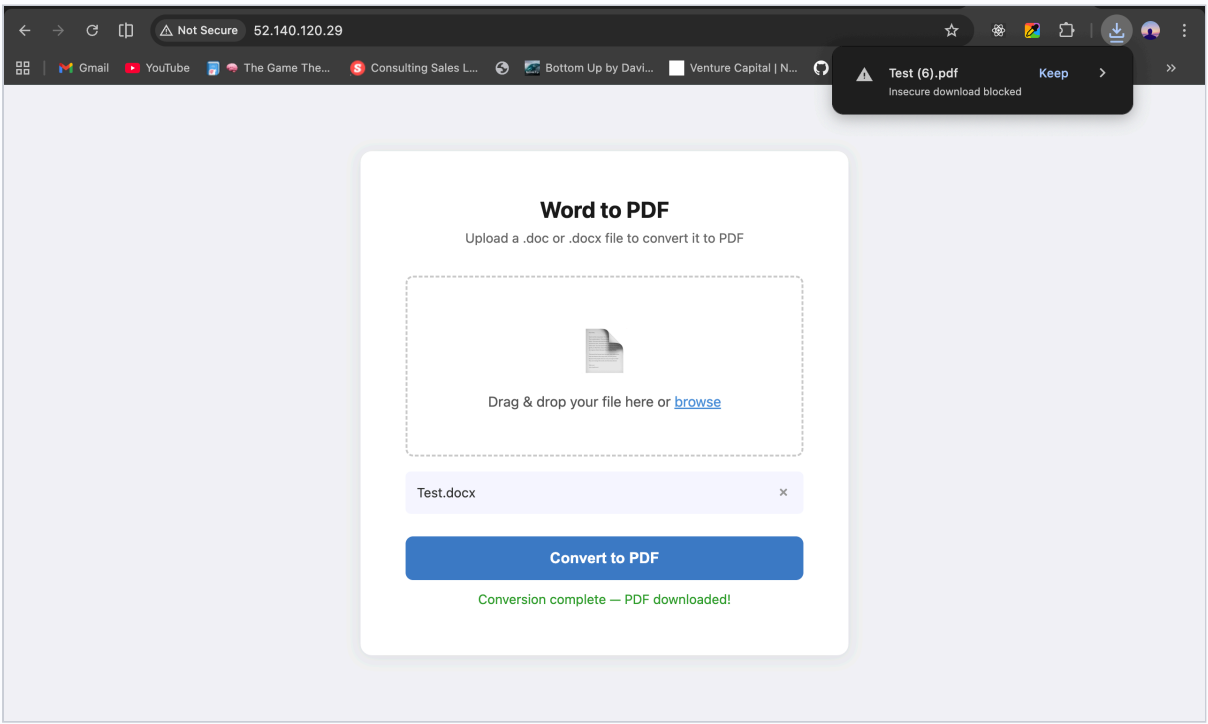

Figure 6: Generated PDF opened in browser viewer.

11. Screenshots with Explanations (Page 4)

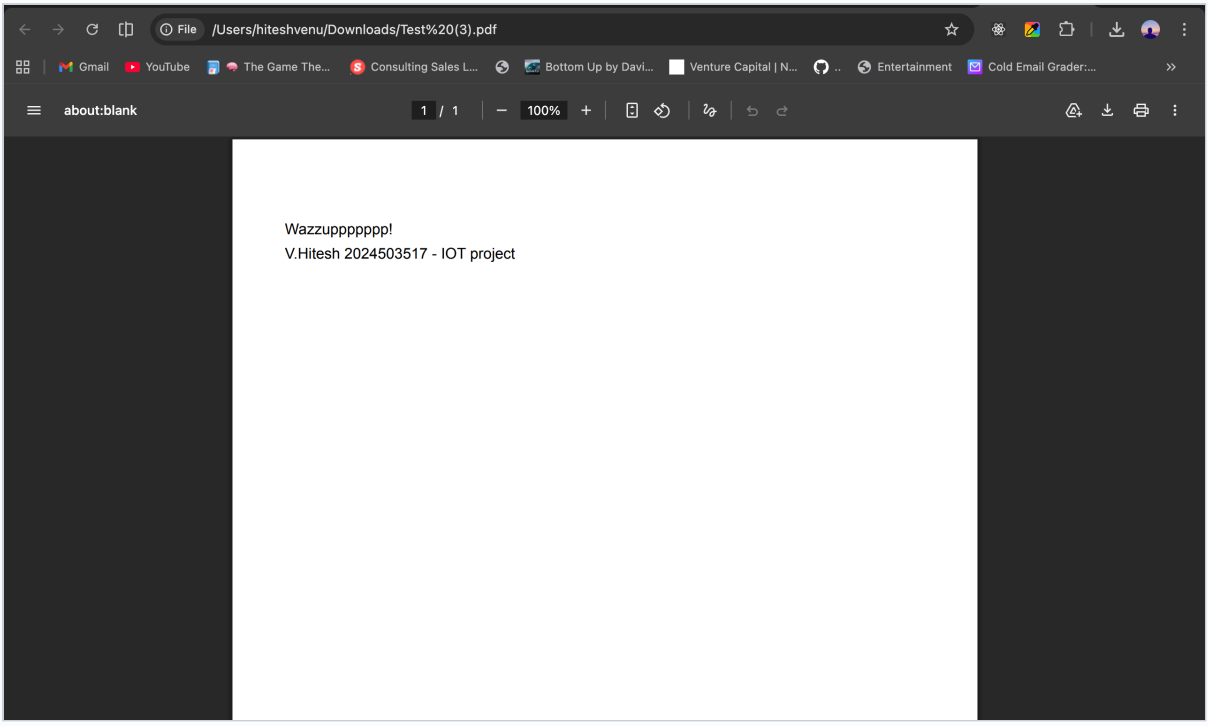

Figure 7: Additional workflow verification screenshot.

12. Challenges and Limitations

- Complex Word formatting may vary in rendered output.
- Insecure HTTP deployments can trigger browser download warnings.

13. Future Enhancements

- Batch conversion and user history.
- Cloud storage integration and PDF customization options.

14. Conclusion

The project delivers a complete and practical Word-to-PDF conversion workflow with a user-friendly interface and stable backend conversion pipeline.

15. References

- Node.js, Express.js, Multer, Mammoth, and Puppeteer documentation.